

Widget Work Flow

HIGH-LEVEL ARCHITECTURE

Primary Path (Agora Call)

User Mic

- Agora
- Backend audio subscription
- STT
- LLM
- TTS
- Agora return stream
- User Speaker

Transcript:

Backend → WebSocket → Widget

Fallback Path (Custom WebRTC)

User Mic

- WebRTC
- Backend
- STT
- LLM
- TTS
- WebRTC audio back
- User

Transcript:

Backend → WebSocket → Widget

COMPLETE WORKFLOW (STEP-BY-STEP)

STEP 1 — User Clicks “Start Call”

Widget does:

1. Open WebSocket connection
2. Create call session on backend
3. Backend returns:
 - sessionId
 - agoraToken
 - fallbackConfig

● **STEP 2 — Try Agora First (Primary)**

Widget:

1. Initialize Agora SDK
2. Join channel with token
3. Publish microphone
4. Subscribe to AI audio stream

Agora handles:

- NAT
- Audio routing
- Scaling

● **STEP 3 — Backend Subscribes to User Audio**

Your backend:

- Joins same Agora channel as a bot
- Subscribes to user's audio stream
- Sends audio frames to STT

Agora provides server-side SDK for this.

● **STEP 4 — Speech-to-Text (Backend)**

Backend streams audio to STT service like:

- OpenAI Whisper
- Deepgram

Streaming mode returns:

- Partial transcript
- Final transcript

Backend sends transcript via WebSocket:

STEP 5 — AI Response Pipeline

When final transcript is ready:

1. Send text to LLM
2. Generate response
3. Send response to TTS
4. Convert to audio stream

Now publish AI audio back:

- Through Agora (primary path)
- Through WebRTC (fallback path)

Also send AI text via WebSocket:

STEP 6 — Detect Agora Failure

If:

- Token invalid
- Network blocked
- SDK fails to connect

Then:

Switch to fallback mode.

FALLBACK WORKFLOW (Custom WebRTC)

1 WebSocket Signaling

Widget:

- Creates `RTCPeerConnection`
- Sends SDP offer via `WebSocket`

Backend:

- Creates peer connection
- Returns SDP answer

`WebSocket` handles:

- offer
- answer
- ICE candidates

2 Audio Streaming

User mic

→ WebRTC

→ Backend

Backend processes exactly same as Agora path:

STT → LLM → TTS

AI audio

→ WebRTC track

→ Browser

Transcript still goes through `WebSocket`.

COMPLETE MESSAGE FLOW SUMMARY

WebSocket Handles:

- Create call
- Session state
- Transcript updates
- AI text
- Fallback signaling

Agora Handles:

- Primary audio transport

WebRTC Handles:

- Backup audio transport

Final Production-Level Flow

1. User clicks call
2. WebSocket session created
3. Try Agora
4. Stream audio
5. Backend does STT
6. Transcript sent via WebSocket
7. LLM response
8. TTS audio streamed back
9. If Agora fails → switch to WebRTC
10. Continue same AI pipeline